



Programación de Bases de Datos PL/SQL sobre Oracle

Índice

1. Introducción	1
2. Preparación del entorno	1
3. Código PL/SQL de ejemplo.....	2
4. Ejercicios de PL/SQL.....	14
5. Estudio a realizar	15

1. Introducción

En esta sesión se continúa usando el entorno de trabajo de las sesiones de prácticas anteriores. En este caso, no es preciso añadir nuevo software, puesto que simplemente se utilizará el gestor de base de datos ya instalado (Oracle Express Edition) y el entorno que facilita toda la gestión de consultas (SQL Developer).

- Oracle Database Express Edition - (*Ya instalado en prácticas anteriores*)
- Oracle SQL Developer - (*Ya instalado prácticas anteriores*)

2. Preparación del entorno

Para esta práctica se usa la misma base de datos que se desarrolló en sesiones prácticas anteriores.

Si se hubiesen borrado las tuplas y/o las tablas tras desarrollar las prácticas anteriores, habría que volver a crearlas e insertar las tuplas correspondientes, mediante los archivos facilitados en la práctica inicial. Así, se tendrán las siguientes tablas sobre la que trabajar:

```
SELECT * FROM sectores;  
SELECT * FROM poblacion;
```

CODS	NOMBRES	PORCENTS	INGRESOSS
1	Agricultura y pesca	0	0
2	Industria y energía	0	0
3	Servicios	0	0
4	Construcción	0	0

DNI	NOMBRE	APELLIDO1	APELLIDO2	FECHANAC	DIRECCION	CP	S
INGRESOS	GASTOSFIJOS	GASTOSALIM	GASTOSROPA	SECTOR			
123456789	Carlos	Gutiérrez	Pérez	15/11/97	Pz. Colón	06300 H	15000
4000	4000	3000	1				
777888999	Gerardo	Martín	Duque	12/12/61	C. del Atún	06002 H	23000
6000	5000	4000	1				

Programación de Bases de Datos

222333444	Pedro	Sánchez	González	01/02/60	C. Ancha	06300	H	22000
5000	3000	2000	1					
333444555	María	García	Gil	10/04/71	C. Diagonal	06400	M	19500
3000	3000	3000	1					
666884444	Ignacio	Costa	Burgos	12/12/82	C. Descubrimiento	10005	H	37000
5000	4500	2500	1					
555666777	Vicente	Marañón	Fernández	15/11/78	Pz. América	10600	H	46000
8000	5000	4000	2					
666777888	Beatriz	Losada	Gijón	12/04/74	Av. Principal	06400	M	50000
15000	8000	5000	2					
888999000	Fernando	Huertas	Martínez	30/05/71	C. Mayor	06002	H	70000
20000	8000	4500	2					
999000111	Francisco	Fernández	Merchán	12/03/79	C. Poniente	10800	H	63000
12000	7500	4500	2					
666999333	Paula	Ordóñez	Ruiz	01/02/90	C. Atlántico	06800	M	25000
10000	3000	2000	2					
987654321	Eva	Moreno	Pozo	10/05/74	C. Justicia	10005	M	40000
9000	6000	3000	3					
111000111	Antonio	Muñoz	Hernández	01/07/89	C. Constitución	06800	H	25000
6500	3500	4000	3					
111000222	Sara	Gálvez	Montes	07/04/73	C. Cádiz	10300	M	40000
11000	9500	6500	3					
111000333	Cristina	Corral	Palma	12/05/76	C. Ermita	10600	M	48000
13000	7800	5200	3					
111222333	Susana	Ruiz	Méndez	22/06/99	Av. Libertad	10800	M	18000
5000	4500	2500	3					
444555666	Ángel	Montero	Salas	07/04/00	C. Tranquilidad	10300	H	14000
3000	3000	3000	4					
888777666	Manuel	Vega	Zarzal	23/11/76	Pz. Azul	10005	H	36000
12000	6000	3000	4					
333445555	Margarita	Guillén	Campos	19/03/74	Av. Héroes	06800	M	50000
12000	7500	6500	4					
222447777	Fermin	Hoz	Torres	08/08/88	C. Curva	06002	H	25000
4000	3000	2500	4					

19 filas seleccionadas.



Todas las explicaciones relativas al código de ejemplo de este guion se han ofrecido en las sesiones de teoría de la asignatura, por lo que no se repiten aquí. Esta documentación básicamente ilustra el código fuente y las salidas que origina, y no debe seguirse como ejemplo para la entrega que cada uno debe realizar: en las entregas de los estudiantes debe explicarse claramente cada acción del código que se entregue.

3. Código PL/SQL de ejemplo

A continuación, se presentan algunos de los ejemplos que se han visto en las clases de teoría.

Variables, tipos, comentarios, mostrar datos...

Como primer ejemplo, se presenta el código de las clases de teoría, incluido en el archivo [PLSQL_01.sql](#). Principalmente se ilustra la declaración de variables, de tipos, comentarios, la forma de mostrar datos en pantalla...

```
SET SERVEROUTPUT ON
DECLARE
  v NUMBER := 2;
BEGIN
  DBMS_OUTPUT.ENABLE;
  v:=v*2;
```



Programación de Bases de Datos

```
DECLARE
  z NUMBER := 3;
BEGIN
  z:=v*3;
  DBMS_OUTPUT.PUT_LINE(v); --escribe 4
  DBMS_OUTPUT.PUT_LINE(z); --escribe 12
END;
DBMS_OUTPUT.PUT_LINE(v*2); --escribe 8
--DBMS_OUTPUT.PUT_LINE(z); --error, ya no es accesible
END;
```

Salida de Script

```
4
12
8
```

En el archivo [PLSQL_02.sql](#) se incluye un nuevo ejemplo que ilustra la **declaración de variables** del tipo de algún atributo de la tabla POBLACION, haciendo uso de **%TYPE**.

```
SET SERVEROUTPUT ON
DECLARE
  ingresos_min poblacion.ingresos%TYPE;
  ingresos_max poblacion.ingresos%TYPE;
  ingresos_med poblacion.ingresos%TYPE;
  total_pob NUMBER(10,2);
  mensaje VARCHAR2(300);

BEGIN
  DBMS_OUTPUT.ENABLE;
  SELECT min(ingresos),max(ingresos),avg(ingresos),count(*) INTO ingresos_min, ingresos_max,
  ingresos_med, total_pob
  FROM poblacion;

  mensaje:='ingresos máximo: '||ingresos_max||' - ingresos mínimo: '||ingresos_min||' -
  ingresos medio: '||ingresos_med||' - total poblacion: '||total_pob;

  DBMS_OUTPUT.PUT_LINE(mensaje);
END;
```

Salida de Script

```
ingresos máximo: 70000 - ingresos mínimo: 14000 - ingresos medio: 35078,95 - total poblacion: 19
```

Funciones, procedimientos, sentencias de bifurcación y estructuras de control

En el archivo [PLSQL_03.sql](#) se incluyen los ejemplos que ilustran la creación de **funciones** y **procedimientos**. En ellos, se muestran ejemplos de **estructuras de bifurcación** (**if**, **if/else**, **if/elsif/else**, **case**) y **estructuras de control** (**loop**, **while**, **for**, **for/reverse**).

```
SET SERVEROUTPUT ON

DECLARE
  mensaje VARCHAR(100);

/* -----
-- */

-- Ejemplo: Funcion para encontrar el mayor de 3 numeros
function Mayor3_1 (n1 number, n2 number, n3 number) return number is
  mayor number := 0;
begin
  DBMS_OUTPUT.PUT_LINE('Mayor3_1');
  if (n1>mayor) then
```



Programación de Bases de Datos



```
        mayor := n1;
    end if;
    if (n2 > mayor) then
        mayor := n2;
    end if;
    if (n3 > mayor) then
        mayor := n3;
    end if;
    return mayor;
End;

/* ----- */

-- Ejemplo: Funcion para encontrar el mayor de 3 numeros

function Mayor3_2 (n1 number, n2 number, n3 number) return number is
    mayor number;
begin
    DBMS_OUTPUT.PUT_LINE('Mayor3_2');
    if (n1 > n2) then
        mayor := n1;
    else
        mayor := n2;
    end if;
    if (n3 > mayor) then
        mayor := n3;
    end if;
    return mayor;
End;

/* ----- */

-- Ejemplo: Clasifica ingresos de poblacion

function Clasif_ingresos (p_codigo poblacion.dni%type)
return varchar2 is
    v_importe poblacion.ingresos%type;
    v_mensaje varchar2(30);
begin
    DBMS_OUTPUT.PUT_LINE('ClasifImp');
    select ingresos into v_importe from poblacion where dni = p_codigo;
    if (v_importe <= 25000) then
        v_mensaje := 'ingresos bajo';
    elsif (v_importe <= 50000) then
        v_mensaje := 'ingresos regular';
    else
        v_mensaje := 'ingresos bueno';
    end if;
    v_mensaje := to_char(v_importe) || ' - ' || v_mensaje;
    return v_mensaje;
end;

/* ----- */

-- Ejemplo: Clasifica un numero

function ClasifNum(n number) return varchar2 is
    respuesta varchar2(30);
begin
    DBMS_OUTPUT.PUT_LINE('ClasifNum');
    case n
        when 1 then
            respuesta := 'Campeón';
        when 2 then
            respuesta := 'Subcampeón';
        else
            respuesta := 'Resto';
        end case;
    return respuesta;
End;

/* ----- */
```



Programación de Bases de Datos



-- Ejemplo: Clasifica ingresos de poblacion

```
function Clasif_ingresos2(p_codigo poblacion.dni%type)
return varchar2 is
    v_mensaje varchar2(40);
    v_importe poblacion.ingresos%type;
begin
    DBMS_OUTPUT.PUT_LINE('ClasifImp2');
    select ingresos into v_importe from poblacion where dni = p_codigo;
    case
        when (v_importe > 0 and v_importe <= 25000) then
            v_mensaje := 'ingresos bajo';
        when (v_importe > 25000 and v_importe <= 50000) then
            v_mensaje := 'ingresos regular';
        when (v_importe > 50000) then
            v_mensaje := 'ingresos bueno';
        else
            v_mensaje := 'Caso Desconocido';
    end case;
    v_mensaje := to_char(v_importe) || ' - ' || v_mensaje;
    return v_mensaje;
end;

/* ----- */
```

-- Ejemplo: Factorial de un número

```
function Factorial1 (n number) return number is
    f number := 1;
    cont number := n;
begin
    DBMS_OUTPUT.PUT_LINE('Factorial1');
    loop
        f := f * cont;
        cont := cont - 1;
        exit when (cont=0);
    end loop;
    return f;
end;

/* ----- */
```

-- Ejemplo: Escribe tabla de multiplicar

```
procedure TablaMulti1 (n number) is
    k number := 1;
begin
    DBMS_OUTPUT.PUT_LINE('TablaMulti1');
    dbms_output.put_line('Tabla de multiplicar del '||n);
    while (k<=10) loop
        dbms_output.put_line(n || ' x ' || k || ' = ' || n*k);
        k := k + 1;
    end loop;
End;

/* ----- */
```

-- Ejemplo: Factorial de un número

```
function Factorial2 ( n number ) return number is
    f number := 1; -- No es necesario inicializar ni declarar k
begin
    DBMS_OUTPUT.PUT_LINE('Factorial2');
    for k in 1 .. n loop
        f := f * k;
    end loop;
    return f;
End;

/* ----- */
```

-- Ejemplo: Tabla de multiplicar, en orden inverso



Programación de Bases de Datos

```
procedure TablaMulti2 ( n number ) is
begin
    DBMS_OUTPUT.PUT_LINE('TablaMulti2');
    for k in reverse 1 .. 10 loop
        dbms_output.put_line(n || ' x ' || k || ' = ' || n*k);
    end loop;
end;

/* ----- */
/* ----- */
/* ----- */
/* ----- */
/* ----- */

BEGIN
    --DBMS_OUTPUT.ENABLE;

    mensaje:='Funcion Mayor3_1(13,08,30): ' || Mayor3_1 (13,08,30);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion Mayor3_2(13,08,30): ' || Mayor3_2 (13,08,30);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion Mayor3_2(13,08,30): ' || Mayor3_2 (13,08,30);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion Clasif_ingresos('555666777'): ' || Clasif_ingresos('555666777');
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion ClasifNum(1): ' || ClasifNum(1);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion ClasifNum(2): ' || ClasifNum(2);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion Clasif_ingresos2('555666777'): ' || Clasif_ingresos2('555666777');
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion Factorial1(5): ' || Factorial1(5);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion TablaMulti1(8): ';
    DBMS_OUTPUT.PUT_LINE(mensaje);
    TablaMulti1(8);

    mensaje:='Funcion Factorial2(5): ' || Factorial2(5);
    DBMS_OUTPUT.PUT_LINE(mensaje);

    mensaje:='Funcion TablaMulti2(7): ';
    DBMS_OUTPUT.PUT_LINE(mensaje);
    TablaMulti2(7);
END;
```

Salida de Script

```
Mayor3_1
Funcion Mayor3_1(13,08,30): 30
Mayor3_2
Funcion Mayor3_2(13,08,30): 30
Mayor3_2
Funcion Mayor3_2(13,08,30): 30
ClasifImp
Funcion Clasif_ingresos('555666777'): 46000 - ingresos regular
ClasifNum
Funcion ClasifNum(1): Campeón
ClasifNum
Funcion ClasifNum(2): Subcampeón
ClasifImp2
Funcion Clasif_ingresos2('555666777'): 46000 - ingresos regular
Factorial1
Funcion Factorial1(5): 120
```



```
Funcion TablaMulti1(8):
TablaMulti1
Tabla de multiplicar del 8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80
Factorial2
Funcion Factorial2(5): 120
Funcion TablaMulti2(7):
TablaMulti2
7 x 10 = 70
7 x 9 = 63
7 x 8 = 56
7 x 7 = 49
7 x 6 = 42
7 x 5 = 35
7 x 4 = 28
7 x 3 = 21
7 x 2 = 14
7 x 1 = 7
```

Funciones y procedimientos con valores de entrada y de salida

En [PLSQL_04a.sql](#) se ilustra el uso de parámetros de entrada y de salida. Se crea una función `suma_A` que recibe dos enteros y devuelve su suma en el `return` de la función. Esta función puede llamarse sin problemas desde el bloque principal, como se puede observar abajo del todo en el ejemplo. Por su parte, en procedimiento `suma_B` tiene tres parámetros, dos de entrada y un tercero de salida, que contiene la suma. En este caso, al haber parámetros de salida no es necesario especificar un valor de retorno.

```
SET SERVEROUTPUT ON
```

```
DECLARE
```

```
    mensaje VARCHAR2(100);
    x1 NUMBER;
    x2 NUMBER;
    x3 NUMBER;
```

```
/* ----- */
-- Ejemplo: Funcion para sumar dos numeros
FUNCTION suma_A (n1 NUMBER, n2 NUMBER) RETURN NUMBER IS
    resultado NUMBER:=0;
BEGIN
    resultado := n1 + n2;
    DBMS_OUTPUT.PUT_LINE('LOCAL - suma_A: ' || n1 || ' + ' || n2 || ' = ' || resultado);
    RETURN resultado;
END;
```

```
/* ----- */
-- Ejemplo: Funcion para sumar dos numeros
PROCEDURE suma_B (n1 IN NUMBER, n2 IN NUMBER, resultado OUT NUMBER) IS
BEGIN
    resultado := n1 + n2;
    DBMS_OUTPUT.PUT_LINE('LOCAL - suma_B: ' || n1 || ' + ' || n2 || ' = ' || resultado);
END;
```

```
/* ----- */
```



BEGIN

```
x1:=12;
x2:=6;
mensaje:='PRINCIPAL - Funcion suma_A: ' || x1 || ' + ' || x2 || ' = ' || suma_A(x1,x2);
DBMS_OUTPUT.PUT_LINE(mensaje);

x1:=13;
x2:=43;
suma_B (x1,x2,x3);
mensaje:='PRINCIPAL: Funcion suma_B ' || x1 || ' + ' || x2 || ' = ' || x3;
DBMS_OUTPUT.PUT_LINE(mensaje);
```

END;

Salida de Script

```
LOCAL - suma_A: 12 + 6 = 18
PRINCIPAL - Funcion suma_A: 12 + 6 = 18
LOCAL - suma_B: 13 + 43 = 56
PRINCIPAL: Funcion suma_B 13 + 43 = 56
```

Funciones y procedimientos persistentes

El fichero [PLSQL_04b.sql](#) se corresponde con una versión del ejemplo anterior ([PLSQL_04a.sql](#)) donde las funciones y procedimientos se crean y almacenan de forma persistente, de modo que, si no se eliminan (con el correspondiente `DROP...`), permanecerían en el espacio del usuario para usos posteriores. OJO, que al definirse varios entornos diferentes **es obligatorio separarlos mediante la barra /**, para que se pueda ejecutar el script sin problemas.

SET SERVEROUTPUT ON

FUERA DEL BLOQUE DE CÓDIGO

```
/* ----- */
-- Ejemplo: Funcion para sumar dos numeros
CREATE OR REPLACE FUNCTION suma_A (n1 NUMBER, n2 NUMBER) RETURN NUMBER IS
    resultado NUMBER:=0;
BEGIN
    resultado := n1 + n2;
    DBMS_OUTPUT.PUT_LINE('LOCAL - suma_A: ' || n1 || ' + ' || n2 || ' = ' || resultado);
    RETURN resultado;
END;
/
```

```
/* ----- */
-- Ejemplo: Funcion para sumar dos numeros
CREATE OR REPLACE PROCEDURE suma_B (n1 IN NUMBER, n2 IN NUMBER, resultado OUT NUMBER) IS
BEGIN
    resultado := n1 + n2;
    DBMS_OUTPUT.PUT_LINE('LOCAL - suma_B: ' || n1 || ' + ' || n2 || ' = ' || resultado);
END;
/
```

```
DECLARE
    mensaje VARCHAR2(100);
    x1 NUMBER;
    x2 NUMBER;
    x3 NUMBER;
/* ----- */
BEGIN
    x1:=12;
    x2:=6;
    mensaje:='PRINCIPAL - Funcion suma_A: ' || x1 || ' + ' || x2 || ' = ' || suma_A(x1,x2);
    DBMS_OUTPUT.PUT_LINE(mensaje);
```




Programación de Bases de Datos



```
x1:=13;
x2:=43;
suma_B (x1,x2,x3);
mensaje:='PRINCIPAL: Funcion suma_B ' || x1 || ' + ' || x2 || ' = ' || x3;
DBMS_OUTPUT.PUT_LINE(mensaje);
END;
/
```

```
DROP FUNCTION suma_A;
DROP PROCEDURE suma_B;
```

Salida de Script

Function SUMA_A compilado
Procedure SUMA_B compilado

```
LOCAL - suma_A: 12 + 6 = 18
PRINCIPAL - Funcion suma_A: 12 + 6 = 18
LOCAL - suma_B: 13 + 43 = 56
PRINCIPAL: Funcion suma_B 13 + 43 = 56
```

Procedimiento PL/SQL terminado correctamente.

Function SUMA_A borrado.
Procedure SUMA_B borrado.

Básicamente, se trata de crear la función y el procedimiento antes del bloque de código (antes del `DECLARE` con el que se inicia el bloque de código). Y, tras el bloque de código, deben incluirse las sentencias `DROP` correspondientes para borrar estas funciones (de no eliminarse, quedarían almacenadas de forma persistente, en el espacio del usuario).

Registros

Para ilustrar el uso de registros, siguiendo con los ejemplos vistos en las clases de teoría, se incluye el archivo [PLSQL_05.sql](#).

```
SET SERVEROUTPUT ON
```

```
DECLARE
    mensaje VARCHAR2(100);
```

```
/* ----- */
```

```
-- Ejemplo: Procedimiento para consultar el nombre e ingresos de una tupla de poblacion
procedure ConsultaPob (pcodigo poblacion.dni%type) is
```

```
    type regproy is record (
        rnombre poblacion.nombre%type,
        ringresos poblacion.ingresos%type
    );
    r regproy;
```

```
begin
    dbms_output.put_line('Procedimiento ConsultaPob');
    select nombre, ingresos into r from poblacion where dni = pcodigo;
    dbms_output.put_line('Nombre: ' || r.rnombre );
    dbms_output.put_line('Ingresos: ' || r.ringresos );
End;
```

```
/* ----- */
```

```
-- Ejemplo: Procedimiento para consultar los datos de un sector
```



Programación de Bases de Datos

```
procedure ConsultaSec(pnumerogestor sectores.codS%type ) is
  r sectores%rowtype;
begin
  dbms_output.put_line('Procedimiento ConsultaSec');
  select * into r from sectores where codS = pnumerogestor;
  dbms_output.put_line('Código sector: ' || r.codS);
  dbms_output.put_line('Nombre sector: ' || r.nombreS);
  dbms_output.put_line('Ingresos sector: ' || r.ingresosS);
end;

/* ----- */
/* ----- */
/* ----- */
/* ----- */
/* ----- */

BEGIN
  DBMS_OUTPUT.ENABLE;

  ConsultaPob('555666777');
  ConsultaSec('3');

  mensaje := 'Fin de programa';
  dbms_output.put_line(mensaje);
END;
```

Salida de Script

```
Procedimiento ConsultaPob
Nombre: Vicente
ingresos: 46000
Procedimiento ConsultaSec
Código sector: 3
Nombre sector: Servicios
Ingresos sector: 0
Fin de programa
```

Cursores

El archivo [PLSQL_06.sql](#) incluye ejemplos vistos en las clases, relacionados con los cursores.

```
SET SERVEROUTPUT ON

DECLARE
  mensaje VARCHAR2(100);

/* ----- */

-- Ejemplo de CURSOR
PROCEDURE EjCursor IS
  CURSOR c_sect IS SELECT * FROM sectores;
  r sectores%ROWTYPE;
BEGIN
  OPEN c_sect;
  FETCH c_sect INTO r;
  CLOSE c_sect;
  dbms_output.put_line('Número sector: ' || r.codS);
  dbms_output.put_line('Nombre sector: ' || r.nombreS);
  dbms_output.put_line('Ingresos: ' || r.ingresosS);
END;

/* ----- */

-- Ejemplo de CURSOR LOOP
PROCEDURE ListarPob IS
  CURSOR c_pob IS SELECT * FROM poblacion;
  r poblacion%ROWTYPE;
BEGIN
```



Programación de Bases de Datos

```
OPEN c_pob;
LOOP
    FETCH c_pob INTO r;
    EXIT WHEN c_pob%notfound;
    dbms_output.put_line(r.dni || ' - ' || r.nombre || ' - ' || r.sector);
END LOOP;
CLOSE c_pob;
END;

/* ----- */
/* ----- */
/* ----- */
/* ----- */
/* ----- */

BEGIN
    DBMS_OUTPUT.ENABLE;

    EjCursor();
    ListarPob();

    mensaje := 'Fin de programa';
    dbms_output.put_line(mensaje);
END;
```

Salida de Script

```
Número sector: 1
Nombre sector: Agricultura y pesca
Ingresos: 0
123456789 - Carlos - 1
777888999 - Gerardo - 1
222333444 - Pedro - 1
333444555 - María - 1
666884444 - Ignacio - 1
555666777 - Vicente - 2
666777888 - Beatriz - 2
888999000 - Fernando - 2
999000111 - Francisco - 2
666999333 - Paula - 2
987654321 - Eva - 3
111000111 - Antonio - 3
111000222 - Sara - 3
111000333 - Cristina - 3
111222333 - Susana - 3
444555666 - Ángel - 4
888777666 - Manuel - 4
333445555 - Margarita - 4
222447777 - Fermín - 4
Fin de programa
```

Excepciones

El archivo [PLSQL_07.sql](#) contiene algunas versiones de los ejemplos expuestos en clases.

```
SET SERVEROUTPUT ON

DECLARE
    mensaje VARCHAR2(100);

/* ----- */

-- Procedimiento para consultar el ingresos de una tupla de poblacion
PROCEDURE FindPob(pcodigo poblacion.dni%TYPE) IS
```



Programación de Bases de Datos

```
v_ingresos poblacion.ingresos%TYPE;

BEGIN
  SELECT ingresos INTO v_ingresos FROM poblacion WHERE dni = pcodigo;
  DBMS_OUTPUT.PUT_LINE( 'FindPob - ingresos: ' || v_ingresos );
  EXCEPTION
    WHEN No_Data_Found THEN DBMS_OUTPUT.PUT_LINE( 'FindPob - EXCEPCION: dni no existe.' );
END;

/* ----- */

-- Permite generar una excepcion (lanzar una excepcion)
PROCEDURE UpdateIngresosPob (pcodigo poblacion.dni%TYPE, p_ingresos poblacion.ingresos%TYPE) IS
  Cont Number;
BEGIN
  SELECT Count(*) INTO Cont FROM poblacion WHERE dni = pcodigo;
  IF (Cont=0) THEN
    RAISE No_Data_Found; -- lanza la excepcion
  END IF;
  UPDATE poblacion SET ingresos = p_ingresos WHERE dni = pcodigo;
  Commit;
  DBMS_OUTPUT.PUT_LINE( 'UpdateIngresosPob - Proceso OK' );
  EXCEPTION -- captura la excepcion
    WHEN No_Data_Found THEN DBMS_OUTPUT.PUT_LINE( 'UpdateIngresosPob - EXCEPCION: dni no
existe.' );
END;

/* ----- */

-- Mismo procedimiento, pero con excepcion de usuario
PROCEDURE UpdateIngresosPob2 (pcodigo poblacion.dni%TYPE, p_ingresos poblacion.ingresos%TYPE) IS
  Cont Number;
  Excep1 EXCEPTION;
BEGIN
  SELECT Count(*) INTO Cont FROM poblacion WHERE dni = pcodigo;
  IF (Cont=0) THEN
    RAISE Excep1; -- lanza la excepcion
  END IF;
  UPDATE poblacion SET ingresos = p_ingresos WHERE dni = pcodigo;
  Commit;
  DBMS_OUTPUT.PUT_LINE( 'UpdateIngresosPob2 - Proceso OK' );
  EXCEPTION -- captura la excepcion
    WHEN Excep1 THEN DBMS_OUTPUT.PUT_LINE( 'UpdateIngresosPob2 - EXCEPCION: dni no existe.' );
END;

/* ----- */

-- Mismo procedimiento, pero con mensaje de error
PROCEDURE UpdateIngresosPob3 (pcodigo poblacion.dni%TYPE, p_ingresos poblacion.ingresos%TYPE) IS
  Cont Number;
BEGIN
  SELECT Count(*) INTO Cont FROM poblacion WHERE dni = pcodigo;
  IF (Cont=0) THEN
    RAISE_APPLICATION_ERROR( -20000, 'UpdateIngresosPob3 - EXCEPCION: dni no existe.' );
  END IF;
  UPDATE poblacion SET ingresos = p_ingresos WHERE dni = pcodigo;
  Commit;
  DBMS_OUTPUT.PUT_LINE( 'UpdateIngresosPob3 - Proceso OK' );
  -- ahora no se atiende la excepcion
END;

/* ----- */
/* ----- */

BEGIN
  DBMS_OUTPUT.ENABLE;

  FindPob('555666777'); -- Ok
  FindPob('AAABBBCCC'); -- Excepcion

  UpdateIngresosPob('555666777',45000); -- Ok
```



Programación de Bases de Datos

```
UpdateingresosPob('AAABBBCCC',45000); -- Excepcion

UpdateingresosPob2('555666777',50000); -- Ok
UpdateingresosPob2('AAABBBCCC',50000); -- Excepcion

UpdateingresosPob3('555666777',55000); -- Ok
UpdateingresosPob3('AAABBBCCC',55000); -- Excepcion

mensaje := 'Fin de programa';
DBMS_OUTPUT.PUT_LINE(mensaje);

COMMIT;
END;
```

A continuación, se incluye el reporte de actividad que ofrece ORACLE. Cabe destacar que **no se muestra el mensaje 'Fin de programa'**, puesto que en el procedimiento UpdateParPer3 el sistema ejecuta una excepción invocando a un mensaje de error del sistema. Por tanto, el programa no continúa: **no imprime el mensaje final de 'Fin de programa'**:

Salida de Script

```
FindPob - ingresos: 46000
FindPob - EXCEPCION: dni no existe.
UpdateingresosPob - Proceso OK
UpdateingresosPob - EXCEPCION: dni no existe.
UpdateingresosPob2 - Proceso OK
UpdateingresosPob2 - EXCEPCION: dni no existe.
UpdateingresosPob3 - Proceso OK
```

Error que empieza en la línea: 3 del comando :

[... CÓDIGO FUENTE ...]

Informe de error -

ORA-20000: UpdateingresosPob3 - EXCEPCION: dni no existe.

ORA-06512: en línea 61

ORA-06512: en línea 88

20000. 00000 - "%s"

*Cause: The stored procedure 'raise_application_error' was called which causes this error to be generated.

*Action: Correct the problem as described in the error message or contact the application administrator or DBA for more information.

Excepciones predefinidas

Excepción	Descripción
ACCESS_INTO_NULL	El programa intentó asignar valores a los atributos de un objeto no inicializado - 6530.
COLLECTION_IS_NULL	El programa intentó asignar valores a una tabla anidada aún no inicializada - 6531.
CURSOR_ALREADY_OPEN	El programa intentó abrir un cursor que ya se encontraba abierto. Recuerde que un cursor de ciclo FOR automáticamente lo abre y ello no se debe especificar con la sentencia OPEN -6511.
DUP_VAL_ON_INDEX	El programa intentó almacenar valores duplicados en una columna que se mantiene con restricción de integridad de un índice único (unique index) -1.
INVALID_CURSOR	El programa intentó efectuar una operación no válida sobre un cursor -1001.
INVALID_NUMBER	En una sentencia SQL, la conversión de una cadena de caracteres hacia un número falla cuando esa cadena no representa un número válido -1722.
LOGIN_DENIED	El programa intentó conectarse a Oracle con un nombre de usuario o password inválido -1017.
NO_DATA_FOUND	Una sentencia SELECT INTO no devolvió valores o el programa referenció un elemento no inicializado en una tabla indexada 100.
NOT_LOGGED_ON	El programa efectuó una llamada a Oracle sin estar conectado -1012.
PROGRAM_ERROR PL/SQL	Tiene un problema interno -6501.



Programación de Bases de Datos

ROWTYPE_MISMATCH	Los elementos de una asignación (el valor a asignar y la variable que lo contendrá) tienen tipos incompatibles. También se presenta este error cuando un parámetro pasado a un subprograma no es del tipo esperado -6504.
SELF_IS_NULL	El parámetro SELF (el primero que es pasado a un método MEMBER) es nulo -30625.
STORAGE_ERROR	La memoria se terminó o está corrupta -6500.
SUBSCRIPT_BEYOND_COUNT	El programa está tratando de referenciar un elemento de una colección indexada que se encuentra en una posición más grande que el número real de elementos de la colección -6533.
SUBSCRIPT_OUTSIDE_LIMIT	El programa está referenciando un elemento de una tabla utilizando un número fuera del rango permitido (por ejemplo, el elemento "-1") -6532.
SYS_INVALID_ROWID	La conversión de una cadena de caracteres hacia un tipo ROWID falló porque la cadena no representa un número -1410.
TIMEOUT_ON_RESOURCE	Se excedió el tiempo máximo de espera por un recurso en Oracle -51.
TOO_MANY_ROWS	Una sentencia SELECT INTO devuelve más de una fila -1422.
VALUE_ERROR	Ocurrió un error aritmético, de conversión o truncamiento. Por ejemplo, sucede cuando se intenta introducir un valor muy grande dentro de una variable más pequeña -6502.
ZERO_DIVIDE	El programa intentó efectuar una división por cero -1476.

4. Ejercicios de PL/SQL

Realizar los siguientes ejercicios:

A) Implementar las funciones o procedimientos necesarios para realizar las acciones básicas sobre bases de datos: **altas, bajas, modificaciones y consultas**. Realizar estas tareas sobre la tabla **Sectores**, de modo que se siga la siguiente secuencia:

- **A.1)** Inserción de un nuevo registro en la tabla (**InsertarSectores(...)**), y ver todos los registros de esa tabla, incluido el que se acaba de insertar (**VerSectores()**).
- **A.2)** Modificación del registro recién insertado (**ModificarSectores (...)**), y ver todos los registros de esa tabla, incluido el modificado (**VerSectores()**).
- **A.3)** Declarar las variables adecuadas:

```
DECLARE
  vnombreS VARCHAR2(20);
  vporcentS NUMBER(5,2);
  vingresoS NUMBER(7,2);
```

Implementar una función o procedimiento de consulta que retorne en estas variables (como parámetros de salida) los valores de Sectores cuyo `codigo` de sector se pase como parámetro de entrada. Por ejemplo, retornar los datos del sector 3.

- **A.4)** Borrado del registro que se ha insertado y modificado, y ver todos los registros de la tabla, donde ya no estará el que se ha insertado (**VerSectores()**).

Por si sirve de ayuda... se ofrece el siguiente guion:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
  vnombreS VARCHAR2(20);
  vporcentS NUMBER(5,2);
```



Programación de Bases de Datos



```
vingresos NUMBER(7,2);
mensaje VARCHAR2(300);

/* ----- */
-- Procedimientos... (A.1, A.2, A.3, A.4)
/* ----- */

BEGIN
  DBMS_OUTPUT.ENABLE;

  -- A.1)
  InsertarSectores (0,'Sin empleo anterior',0.0,0.0);
  VerSectores();

  -- A.2)
  ModificarSectores (0,'Desconocido',0.0,0.0);
  VerSectores();

  -- A.3)
  ConsultarSectores (0,vnombreS,vporcentS, vingresosS);
  dbms_output.put_line('Nombre sector: ' || vnombreS);
  dbms_output.put_line('Porcentaje: ' || vporcentS);
  dbms_output.put_line('Ingresos: ' || vingresosS);

  -- A.4)
  BorrarSectores (0);
  VerSectores();

  mensaje := 'Fin de programa';
  DBMS_OUTPUT.PUT_LINE(mensaje);
END;
```

B) Realizar versiones de los procedimientos anteriores, de modo que se gestionen las excepciones de forma adecuada. Llamar a los procedimientos/funciones con el mismo nombre, pero esta vez terminados en 2 (*InsertarSectores2*, *ModificarSectores2*, *ConsultarSectores2* y *BorrarSectores2*).

5. Estudio a realizar

- Probar el código facilitado y desarrollar los ejercicios propuestos en esta sesión.



EXTRA POINT: Con carácter voluntario, se presenta la posibilidad de obtener un *EXTRA POINT* a quienes deseen profundizar más en los temas presentados en este guion práctico. Para ello, en la memoria a entregar, se deberá incluir (además de los apartados obligatorios del guion) un nuevo apartado denominado *EXTRA POINT* e identificado con el logo *BONUS*, donde se presente algún aspecto novedoso o diferenciador con respecto a la propuesta práctica base.

- **Entregables:** debe entregarse
- **OPCIÓN (1)** – OBLIGATORIO

(1) El código fuente correspondiente a los ejercicios debidamente resueltos.
Los ejercicios deben implementarse y documentarse **para todos los SGBD**

Programación de Bases de Datos

(2) Un PDF en el que se documente:

- Un anexo completo con **las salidas obtenidas tras la ejecución del script completo del código desarrollado**. **Las salidas obtenidas deben incluirse en la documentación en modo texto y no como una captura de imágenes.**

• **OPCIÓN (2)** – OPTATIVO, además del punto (1) obligatorio:

- Todo el **código fuente** desarrollado y entregado en esta sesión práctica, ampliamente **comentado y documentado**, con las explicaciones necesarias y las **salidas parciales** obtenidas por cada fragmento de código. Si alguna función/procedimiento del código fuente es idéntica en todos los SGBD, es suficiente con explicar esa función/procedimiento una única vez. No es preciso documentar los ejercicios resueltos en este guion y que no formen parte del código del estudiante, de modo que solo es preciso comentar las funciones usadas en los ejercicios propuestos. **El código fuente debe incluirse en la documentación en modo texto y no como una captura de imágenes, tal y como se presenta en los guiones prácticos de la asignatura.**
- Particularmente, debe explicarse detalladamente todos los **aspectos novedosos** que incluyen el tema que se presenta.
- Todas las capturas, salidas, código... que se incluyan en la documentación deben ser **perfectamente legibles**, presentándose con la máxima claridad y corrección.